

Informing Housing Policy through Web Automation: Lessons for Designing Programming Tools for Domain Experts

Chris Hess

clh285@cornell.edu

Cornell University

Brooks School of Public Policy

Ithaca, New York, USA

Sarah E. Chasins

schasins@eecs.berkeley.edu

University of California–Berkeley

Electrical Engineering and Computer Sciences

Berkeley, California, USA

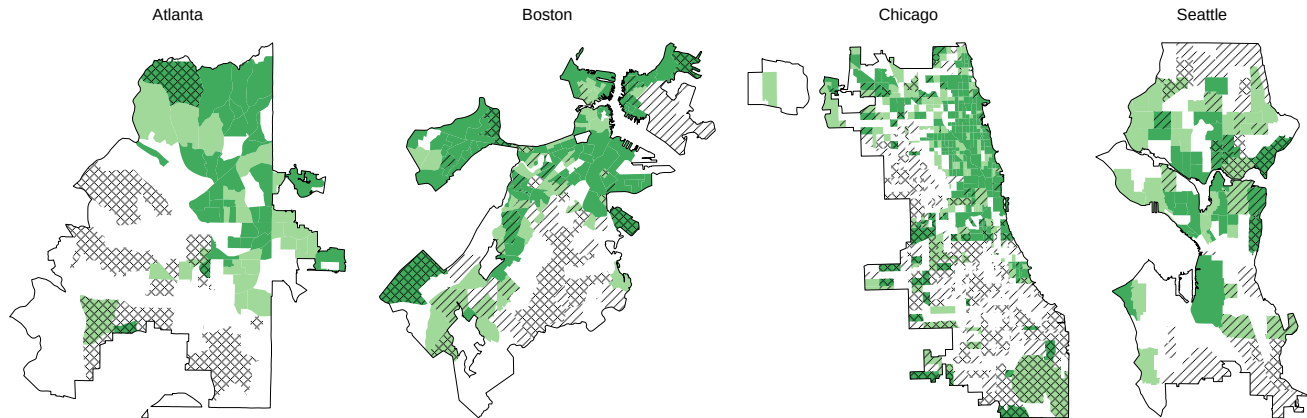


Figure 1: Neighborhood coverage for Zillow and GoSection8, an online platform focused on households receiving housing assistance, within example cities of Atlanta, Boston, Chicago and Seattle. Degrees of underrepresentation (i.e., moderate, high) on Zillow are indicated with hatching and crosshatching, while underrepresentation on GoSection8 is denoted by light green and dark green shading. Across each of these cities, there are distinct coverage gaps such that the neighborhoods one potentially sees are shaped by the platform one uses.

ABSTRACT

Housing costs have risen dramatically in the past decade, surpassing their pre-Recession levels, but the data that housing researchers and policymakers rely on to understand these dynamics remain subject to important limitations in their spatiotemporal granularity or methodological transparency. While these aspects of existing public and private data sources present barriers to understanding the geography of cost and availability in markets across the United States, web data about housing opportunities provide an important alternative—albeit one that demands technical skills that would-be data users may lack. This case study documents the experiences of a collaboration between social and computer scientists focused on using a novel programming-by-demonstration tool for web automation, Helena, to inform rental housing policy and inequalities in the United States. While this project was initially focused on collecting housing ads from a single site within the Seattle area, the

capacity to scale our project to new sources and locations afforded by Helena’s human-centered design allowed a team of social scientists to progress to scraping data across the country and multiple platforms. Using this project as a case study, we discuss a.) important programming and research challenges that were encountered and b.) how Helena’s design helped us overcome these barriers to using scraped web data in basic research and policy analysis.

CCS CONCEPTS

• **Human-centered computing** → *User studies*; • **Information systems** → *Online advertising*; • **Applied computing** → **Sociology**.

KEYWORDS

web automation, housing policy, applied sociology

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

CHI '22 Extended Abstracts, April 29–May 5, 2022, New Orleans, LA, USA

© 2022 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-9156-6/22/04.

<https://doi.org/10.1145/3491101.3503575>

ACM Reference Format:

Chris Hess and Sarah E. Chasins. 2022. Informing Housing Policy through Web Automation: Lessons for Designing Programming Tools for Domain Experts. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts (CHI '22 Extended Abstracts)*, April 29–May 5, 2022, New Orleans, LA, USA. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3491101.3503575>

1 INTRODUCTION

A steady march of price increases across housing markets in the United States underpins a growing housing affordability crisis for many households, with this trend felt most acutely among relatively vulnerable populations like low-income renters since “the rent eats first” [12]. The average home price to income ratio is now almost 5:1 nationwide, nearly half of renters pay more than 30% of their income towards housing, and the number of unhoused persons in the United States is greater than at any other time in recent history [11]. Further, these affordability issues intersect with existing housing and neighborhood inequalities by race and ethnicity, with Black and Latinx households less likely to be homeowners and more likely to be burdened by housing cost compared to non-Latinx white households—even after accounting for relevant household and contextual differences [15] [11].

Understanding the scope and severity of these housing problems and inequalities depends on timely, accurate data about the conditions faced by households searching for housing, with information about cost and availability accordingly important for housing policymakers and researchers alike. Within local governments, data about housing conditions inform growth planning efforts and otherwise provide insight about the need for fair and affordable housing within the legislative agenda [9]. Similarly, non-profit organizations like public housing authorities use information about rental cost and availability to determine rent level that housing vouchers should cover and understand the affordable neighborhoods where socioeconomic mobility is most likely [19] [5]. Finally, data about housing costs and availability are used widely as inputs within social research on topics including but not limited to: variations in household mobility, patterns of residential segregation and the impacts of the COVID-19 pandemic [10] [28] [26].

These data needs create a premium for developing alternative methods of data collection, while the advent of the internet as the predominant source of information about housing opportunities means that much of the important data regarding cost and availability is now more accessible than ever before—albeit in a format that cannot be directly used for analysis. Despite the apparent potential to collect data from sources like Craigslist or Zillow in service of generating policy-relevant, timely information about housing conditions in a given area, perhaps the greatest challenge is most social scientists’ lack of training in the programming areas crucial to web automation [3]. While social science training in data analysis and statistics has come to recognize the value of open-source programming languages like Python and R, there are important differences in the use cases of these languages between social science and web automation. For example, web programming languages are largely absent from training in the social sciences, despite the importance of HTML and JavaScript when reverse engineering pages to find relevant values. Frameworks like Selenium also require general programming skills that may surpass experience with using constructs for iteration, connections or exception handling, particularly in use cases where ongoing execution of a given program is needed to regularly collect data.

While the mismatch between social scientists’ technical training and the domain knowledge needed for web automation is a potential barrier to incorporation of “big data” within social research [21],

this constraint presents an opportunity for computer scientists to work with social scientists to create tools that understand and meet the needs of potential users in this community. This case study is based on an ongoing collaboration between social and computer scientists to inform housing policy through scraping web data from websites like Craigslist using a programming-by-demonstration tool for web automation called Helena [2, 4]. Initially started in service of a report to the City of Seattle about rental conditions and declining rental affordability, this effort has scaled to cover multiple web data sources and all markets in the United States, as the data proved valuable across basic and applied research.

This case study illustrates important insights about the design and research challenges HCI researchers encounter in collaborations with non-technical domain experts, especially in developing programming tools for such users. Along the way, we also discuss aspects of tool design that help meet these challenges, and the scale of the impact that this kind of research can have. In describing this case study, we will occasionally return to a few key design guidelines, themes that appeared repeatedly throughout the five years of this project. Table 1 summarizes these guidelines. We will use the boxed keywords throughout the case study description to highlight situations that motivate these design guidelines.

Contributions. This paper describes design guidelines for end-user programming tools targeted at non-technical domain experts, with an emphasis on the social sciences. The authors have distilled these guidelines from a variety of projects and collaborations over the past decade. In this paper, we describe a long-running, illustrative project that has been active for the past five years. We use this concrete context to exhibit the need for each design guideline.

2 BACKGROUND

This section briefly describes the requisite background knowledge both from sociology and from computer science.

2.1 Sociology

Existing public data available through the decennial Census and American Community Survey (ACS), both products of the Census Bureau, have clear benefits to basic researchers and practitioners interested in studying housing conditions and the rental market more generally, since they are freely available online, well-documented and based on established survey methodologies. The primary challenges with these public data options relate to their spatial and temporal granularity, since small area estimates (e.g., neighborhoods, zip codes) are only published on a decadal or five-year moving average basis. Furthermore, lags between surveying and publication mean that the “freshest” estimates for a given area use responses that are at least one year old and as many as six years old (in the case of the ACS). Finally, the Census Bureau includes households who may have signed their lease any number of years before and simply remained in place, making it even more problematic for understanding the costs that households currently searching for housing would face on the market.

These limitations mean that studying localized changes in housing availability or rent within a relatively short span (e.g., less than 5 years) is methodologically fraught, if not impossible, using the public data sources that academics and practitioners typically rely

1. Design for Easy First Drafts	Design for easy prototyping, especially for users to be able to create a functional first draft quickly. To the extent possible, let users draft working programs <i>without</i> first understanding technical complexity, even if they will eventually benefit from understanding the technical complexity.
2. Design for Scale-Up (Computation)	Design for first drafts to be extended or “productionized” into large-scale and long-running projects.
3. Design for Scale-Down (Human Effort)	Human interaction burdens that are bearable during prototyping may be overwhelming as a project is productionized. Design so that small amounts of additional programming effort can be substituted for large amounts of other human effort.
4. Design for Integration with Other Tools	Prioritize the ability to import, export, and transition inputs between the many tools in the domain experts’ existing toolkits.
5. Design for Single-User Mixed Expertise Levels	Expect users to mix manual and programmatic editing—throughout a project’s lifecycle and user expertise growth. Make both manual and programmatic manipulation available <i>throughout</i> , so users can switch freely between them at any point. Even as users develop technical skills, they continue to face fresh challenges that they can tackle best via the manual and direct manipulation interfaces designed for non-technical users. Users who advance to “technical expert” approaches in their work may occasionally edit their automation processes in “non-expert mode,” when they face new hurdles.

Table 1: Design Guidelines: Usable automation and programming tools for social scientists. A set of design guidelines, refined over several years of the case study described in this paper, as well as a variety of other projects and collaborations. This case study offers an example of how following these design guidelines can yield productive and illuminating HCI-Social Science collaborations.

on for their research, in many cases because they have no alternative. While the lack of timely data about local rent conditions has created an industry of information brokers aiming to provide users with more recent, spatially-granular data, the cost of such data sources and lack of transparency around their generation constitute important limitations to their adequacy for basic and applied research about housing. First, subscription fees create a starting barrier since many researchers simply do not have the resources available to pay typical subscription costs, let alone defray fees for custom tabulations that might be required to study dynamics in a particular geography or time frame where data are not already available for purchase. Second, even when private data brokers have products that meet the needs of researchers in terms of cost, these entities otherwise tend to remain opaque about how any estimates are generated and may still set burdensome restrictions on the ways the data can be used.

For example, in the present case study, the City of Seattle and other non-profit organizations in the area relied on estimates purchased from a real estate firm, Dupre+Scott, whose estimates were based on survey responses from a limited set of larger, corporate landlords. While this sampling strategy was advantageous for Dupre+Scott because they collected survey responses in exchange for sharing results with their corporate participants, the fact that smaller landlords had effectively zero probability of being sampled constituted an important but underappreciated difference compared to estimates from the Census. Further, the strategy of only surveying larger landlords meant that “mom and pop” landlords were effectively outside the survey’s sampling frame, even

if these owners are a significant share of all landlords and were of particular interest to policymakers due to their potential affordability [9] [13]. Absent a better alternative, limitations like these may be palatable, or at least unavoidable, for a given data user—whether they are a graduate student conducting a thesis study or a local government assessing the impact of a policy. Nonetheless, the trade-offs in this example underscore how for-profit data providers typically rely on imperfect, non-probability samples of units for rent, even if marketed in a manner that fails to acknowledge or even obfuscates this fact.

These limitations between public and private data options imply that researchers must weigh trade-offs between timeliness, geographic specificity, and methodological transparency when using typical secondary data sources. Data users can rely on trusted public data like the Census’s American Community Survey (ACS), though they may need to wait years to conduct their research and thereby undermine their study’s value altogether. Should a data user employ more timely private data, they may be able to follow through on research sooner, but results will ultimately be contingent on obtaining organizational consensus about the validity of whatever inputs and methodologies were used to produce estimates—even if those inputs and processes are never fully understood or cannot be known. Altogether, these issues highlight how a third option of primary data collection through web scraping potentially alleviates constraints of cost and transparency, despite imposing separate issues related to technical skills essential to web automation [1]. These considerations are particularly relevant within the context of policy-making and program administration, where additional time

not only implies financial but also potential social costs to inaction (e.g., housing cost burden, eviction).

2.2 Computer Science

2.2.1 The Helena Programming-By-Demonstration Tool. Helena is a Programming-By-Demonstration (PBD) tool for writing web automation programs. It is designed for users interested in web automation who otherwise do not have backgrounds in computer science. Web automation programs serve a number of purposes—e.g., data entry, webpage testing, web auditing; for this collaboration, the most important purpose was web scraping. In particular, this collaboration called for collecting datasets spread over tens of thousands of individual webpages.

The Helena PBD tool is implemented as a browser extension. To draft a program, the user turns on the extension, then demonstrates how to collect the first row of their target dataset as they browse the pages in their web browser. For instance, to collect information about a list of Craigslist posts, the user would demonstrate how to collect information about the first post in the list. Based on this demonstration, Helena writes a program that iterates through the list to collect the same pieces of information about each post. The PBD tool also comes with a custom language, the Helena programming language, and this is the language of the generated programs. The Helena language includes custom constructs for actions that are important in web automation, such as `load`, `click`, and `scrape`. Helena programs are typically displayed in a block-based programming environment. See 2 for a walkthrough of how a user writes a Helena program. The (a) and (b) panels of Figure 2 show a user highlighting relevant webpage elements in their browser as they provide an input demonstration, and then the (c) panel of 2 shows the generated program in the Helena language.

For more details on Helena, its user interaction model, its design process, or its implementation, we refer the reader to [4].

2.2.2 PBD for End Users. Helena builds on the long history of Programming-By-Example (PBE) and PBD tools designed for users from outside of computer science. In PBE, the specification for the program synthesizer is a set of input-output pairs, and the synthesized program should be capable of transforming each input into the corresponding output. For example, with the input-output pairs (3, 7) and (11, 15), the synthesizer might produce the program `i+4`. In PBD, the specification includes the input-output pairs, but also the sequence of actions the user takes to transform each input into the paired output. The potential to reach users outside of computer science via PBE and PBD interfaces has long driven interest in program synthesis [25] [24]. Within the web automation space in particular, efforts to support end-user programming include CoScripter [23], iMacros [17], Vegemite [27], and FlashExtract [22]. Despite the long history of technical progress in this space, assessments of whether PBD and PBE tools work for domain experts outside of computer science are still relatively rare [20]. Prior work in this space reveals that, even in a population of highly experienced programmers, more than 70% of programmers cannot successfully write web automation scripts within an hour for simply-structured webpages [4]. Thus for the purposes of this work, we are especially interested in how PBD supports users in tasks that take experienced programmers on the order of hours rather than minutes.

3 CASE STUDY

3.1 Pilot study on Seattle

Two decades of rising rents and a booming local technology economy have put Seattle in dubious company with other high-cost cities like San Francisco and New York City in terms of housing unaffordability. The American Community Survey estimates for 2015-2019 indicate that Seattle had a median rent of \$1614 for this period, up from \$1,038 in 2000 (adjusted for inflation). In 2016, these trends of rising rental cost resulted in vocal public concerns about the unaffordability of renting in the city, with persons of color and lower-income tenants disproportionately affected by high rents in many gentrifying, previously working-class neighborhoods. This intensifying scrutiny about housing cost, along with broader concerns about discrimination against particular tenants in the application process, ultimately spurred the Seattle City Council to pass ordinances aimed at regulating housing access through mechanisms like banning criminal records as an application criteria and requiring landlords to accept the first qualified application. While these ordinances included mandates to study their effects as part of the legislation, the City lacked the pre-intervention data on landlord behaviors and also did not have the means to test for policy impacts on rental cost given the temporal and spatial granularity of existing data sources.

The City's data needs for program evaluation motivated a rental market study led by the University of Washington, where we were tasked with exploring novel forms of data about the rental market and generating data on rental market practices among landlords. Inspired by a recent project using scraped data to study market-level rental costs [1], we realized that housing advertisements from websites like Craigslist might be a solution to the City's data problem. While there was some trepidation about the legality of scraping given high-profile cases like *Craigslist v. 3Taps* that resulted in fines, we ascertained that not-for-profit uses are considered acceptable relevant statute and case law [6].

However, we quickly recognized that a first barrier to using web data in our data comparison was simply that none of the social scientists on this project had any experience with web scraping. Graduate computing training in the social sciences largely focuses on data analysis using software like Stata or the statistical programming language R, and this background meant that we were unaware of common scraping issues (e.g., scrapers breaking after page changes, IP blocking for excessive server burden), or even common challenges with running code on a continuous basis over time rather than only as needed (e.g., dependency issues, developing and managing useful logs). To handle the challenges of automating web data collection, we adopted the Helena web automation tool, which made it easy to draft programs via PBD (**First Drafts**). We could readily identify the data we wanted in the rendered HTML, so the PBD interface gave us a straightforward way of drafting programs. More generally, Helena allowed us to focus primarily on how we would start to use these novel data rather than dwelling on how to obtain them. We also found the Helena language's skip block construct [2] was important for keeping duplicates at bay, enforcing assumptions about the underlying unit of analysis, and speeding up long-running scrapers (**Scale-Up (Computation)**). This was

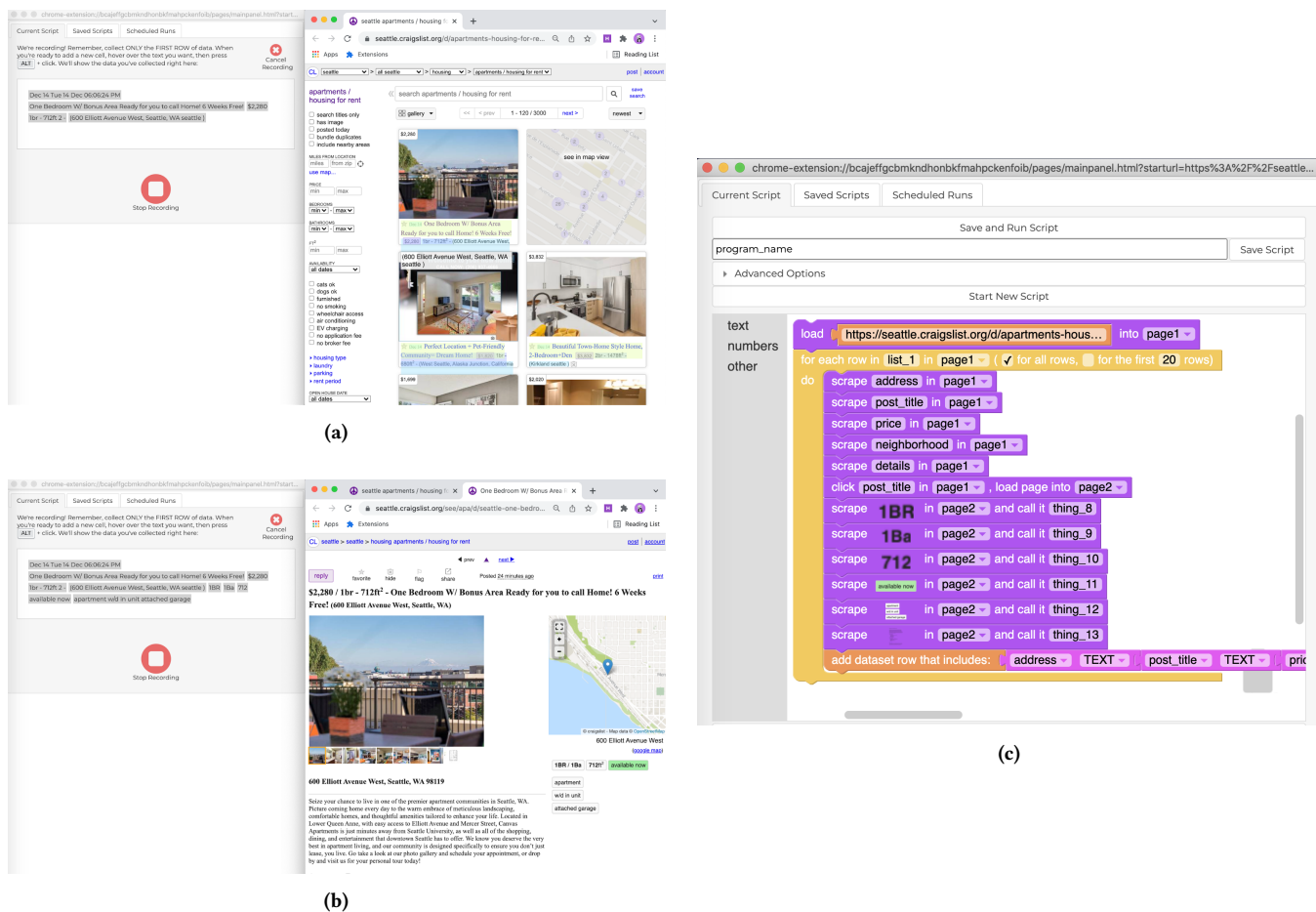


Figure 2: Helena PBD Tool + Helena Programming Language. To draft a Helena program, users browse webpages in a standard browser, augmented with the Helena browser extension. They select webpage elements that their program should collect—(a) and (b). In particular, the user collects the first row of all tables of interest. Note that the elements appear in the control pane at the left as the user selects them from the webpage on the right. A single row may span multiple webpages; this is the case here, where the details of a Craigslist post appear both in the post list and in a post-specific detail page. The Helena program synthesis algorithm then generates a program that collects all rows of all tables of interest—(c). The user may edit the program in the block-based programming environment, included in the browser extension—(c).

particularly important since Craigslist posts’ embedded unique identifiers do not identify unique *vacancies* but rather unique *posts* on the platform.

The social science research challenges at this stage fell into two different categories. The first and more straightforward task was to validate small area rental cost estimates based on the scraped advertisements. Using the most recent data (at the time) from the City’s preferred analytics broker and the Census Bureau’s American Community Survey (ACS), we compared median neighborhood rents estimated from these sources with estimates generated from the Craigslist listings we gathered over approximately two years. Figure 2 shows this analysis from 2018, where there are clear differences between Craigslist and Dupre+Scott in terms of coverage, though the two sources are in general agreement about levels

across most census tracts. Whereas geographic coverage is the primary difference between the Craigslist and Dupre+Scott estimates, comparisons of the Craigslist and Dupre+Scott estimates with public ACS data suggest that the ACS level is far behind Craigslist and Dupre+Scott, despite this public data covering most neighborhoods like Craigslist. While this and other analyses revealed strong similarity between the City’s preferred data and ours in terms of neighborhoods’ relative rankings, our estimates did show that historically diverse South Seattle neighborhoods that were affordable according to other sources had considerably higher rents when analyzed through current market listings.

Even though we had reasonable explanations for this observation, this issue was emblematic of our other research challenge related to sufficiently communicating the potential advantages of scraped data over conventional approaches. Namely, our estimates

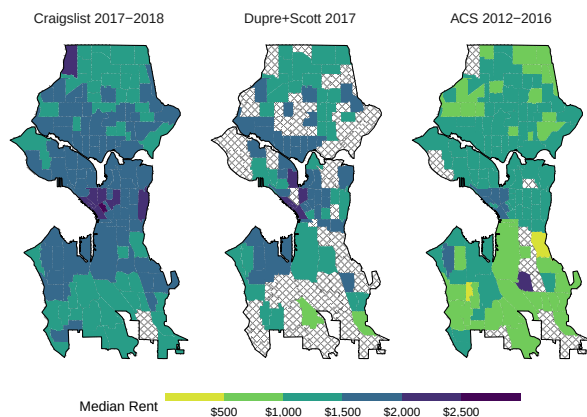


Figure 3: Estimates for median rent asked among 1 bedroom rental units for neighborhoods (i.e., census tracts) in the City of Seattle, across Craigslist, Dupre+Scott and the 2012–2016 American Community Survey (ACS). Neighborhoods with missing data indicated with crosshatching.

suggested an even bleaker picture regarding affordability in neighborhoods with greater racial and ethnic representation and lower median household incomes—areas where the Dupre+Scott data already in use were even known to have significant sampling limitations. Despite our survey of landlords also showing that the vast majority of landlords in the Seattle do in fact advertise vacancies with Craigslist [9], we were ultimately unsuccessful at encouraging the adoption of our data among some departments of the City of Seattle. Nevertheless, housing authorities for the Seattle area have used our estimates in ongoing research since this pilot study, suggesting that there were important differences in how the empirical narrative we shared was received between the two groups of data users. One explanation for the varying reactions stems from the observation that our information had different potential implications for the City vs. for the housing authorities (e.g., political blowback related to unaffordability vs. wanting to support families by raising how much a housing voucher covers). Regardless of the reason, these translational challenges did inform a revised methodology for estimating rents in a given area ([Integration](#)). Specifically, by weighting scraped ads according to ground-truth data about the stock of rental opportunities in an area, we can support inferences to a target population beyond the sampling frame, much like other recent studies have done with different non-probability samples generated from online user activity [32] [31].

3.2 Scaling to national coverage

The second phase of our project focused on expanding the system we had developed for scraping Craigslist rental advertisements to cover the largest 100 metropolitan areas by population in the United States ([Scale-Up \(Computation\)](#)). We began with an expansion to just two additional locations (Ft. Lauderdale and San Antonio) in order to support a policy analysis that was eventually published in the Department of Housing and Urban Development’s *Cityscape* journal. This study compared our rent estimates to those from the

Census and private data providers like Zillow and CoStar in order to inform a rule change related to defining how much vouchers should reasonably cover for each zip code, rather than for much larger areas like counties or metropolitan areas as existing policy dictated [16]. Our analysis of scraped, commercial and public data sources indicated that the underlying data for implementing these “small area fair market rents” had substantial limitations for capturing current market conditions and would also create potential externalities (i.e. smaller subsidy) for households who choose not to relocate.

After this pilot expansion, we turned our focus to scraping Craigslist listings from the largest 100 metropolitan areas in the U.S., with the key technical challenges at this stage centering on how to manage many variations of our scraper as well as how to process and store the resulting data given the greater volume ([Scale-Down \(Human Effort\)](#)). Quantitative social science projects typically focus on running a single version of a script on a single computer, so creating a system for different configurations of a general process and managing the execution of this program across different machines were relatively new programming tasks for our team. The key for addressing this challenge was to separate the program drafting and program deployment stages. Program drafting (with the PBD tool) remained limited to the teams’ personal laptops. Meanwhile, we used a Docker image running Helena programs in a headless browser ([Scale-Up \(Computation\)](#)) to execute large numbers of variants of the PBD-written programs. For a given website, we first generated a scraper for a single location ([First Drafts](#)), then we manually generated about one hundred variants of this program with different URLs to account for different locations of interest, and then we mapped identifiers for these programs to a YAML file with the hostname of the machine where we would run the program regularly. This process became the basis for running Helena programs that covered all relevant locations through Docker containers spread across a set of four virtual machines (VMs). While this approach achieved our goal of scraping at greater scale, the amount of manual processing required on our part was prohibitive and this would ultimately prompt changes to Helena as well as our workflow for using it.

The amount of manual program editing was especially a problem because the Craigslist page structure occasionally changed, with these page structure changes necessitating a large scale revision to the program drafting side of our process after months of successful deployment. While Helena programs can adapt to moderate webpage redesigns, substantial changes to the webpages’ underlying DOM trees can prevent them from finding the target data. When this happened, we could demonstrate a single new Helena program quickly. Nonetheless, given the national scale we were interested in scraping at, painstaking manual modification of our full configuration list of about one hundred variants was still quite slow ([Scale-Down \(Human Effort\)](#)). This pain point informed later refinements of the Helena tool, which allowed explicitly parameterizing Helena programs and passing arguments to the Dockerized programs from the command line—e.g., passing a URL to control where a given scraper started its scraping. During instances when our VMs’ data collection appeared to halt, we frequently returned to the Helena GUI on our own machines to determine whether page

changes, dependencies, IP blocking or VM networking issues were at fault ([Single-User Mixed Expertise Levels](#)).

Once we had a working system for scraping data at national scale, we also realized that our prior data processing for just a few locations would be a much longer ordeal with the volume of data our expanded system was collecting. Moreover, we found that there were considerable differences in data formatting and precision between metropolitan areas that required retooling existing processing and geocoding to account for these new aspects of the scraped data. By running our processing in parallel and storing the results of each process in a database, we eventually ironed out these wrinkles and arrived at a storage solution for spatial data based on PostgreSQL ([Integration](#)) where we could query aggregates we needed often (e.g. median rent for a given neighborhood or city) directly from our database, alleviating the need to repeatedly execute the processes by hand in software like R or ArcGIS [7] [29] [30]. Again, this relates to [Scale-Down \(Human Effort\)](#), letting users trade off additional programming burden for additional human effort.

At this stage in our project's development, we also started to experiment with analysis of data features beyond the core parameters of a housing unit like its rent asked and square footage. The key feature in this case was the descriptive text that landlords author for a given unit, which a colleague of ours used in a co-authored study about differences in how housing opportunities are described between neighborhoods of varying racial and ethnic diversity [18]. This work highlighted how the digital realm is not independent of the existing structure of inequalities in the built and social environment, but instead is a context where discourse about things like neighborhoods and housing has systematic differences based on the populations that reside there. In contrast to our experience sharing estimates with the local City officials in Seattle, the research we conducted during this stage informed how both academic and Federal housing policy audiences were more interested in the potential contributions of web data to housing research, with folks from these research environments more cognizant about the strengths and limitations of different estimates and also less concerned with the political consequences for the picture that the data painted.

3.3 Scaling to multiple platforms

The most recent stage of our project expanded the number of scrapers running on a given day from the hundreds to the thousands, with this scraping system covering all urbanized areas in the United States as well as five different sources of listings (Craigslist, Apartments.com, GoSection8, Trulia, Zillow) ([Scale-Up \(Computation\)](#)). With more sources and locations to manage, we had to revisit our prior single text file approach to managing our configurations, in addition to being more efficient about how we deployed containers to the computers that ran our scrapers (i.e., reducing workload imbalance across our VMs). Since we were already working with Docker and had a set of virtual machines running our Helena programs, we decided to implement our expanded scraping within a Kubernetes cluster ([Integration](#)) where pods for a given location and source combination are scheduled using the Cronjob API, namespaces define container-level and source-level resource quotas, and a new

parameterization feature of Helena allowed us to demonstrate a single program for each source of interest and customize it pragmatically for all cities of interest ([Scale-Down \(Human Effort\)](#)).

Although we were unfamiliar with Kubernetes at first, our experience with Cron, Docker and YAML from the prior stage supported us in learning the orchestration framework and identifying what it could offer in terms of allocating containers more evenly across VMs. For example, while the concept of namespaces was initially unfamiliar to the team at this stage, through testing the team nonetheless recognized how using a separate namespace for each source could provide a method for dynamically adjusting when and where containers would be deployed for a given source ([Scale-Down \(Human Effort\)](#)). This first change was a significant improvement over our prior implementation, since it allowed for programs to run for as long as they needed and all VMs to work at roughly comparable levels of CPU and memory pressure. Second, and perhaps more importantly, we recognized that Helena's parameterization feature could mean only ever demonstrating a single program—rather than hundreds or thousands—when pages inevitably changed enough that our existing demonstrations failed ([Scale-Down \(Human Effort\)](#)). Working backwards from the user interfaces and URLs of the websites we wanted to process, we determined how we could use public data (e.g. FIPS codes, County/State names) or an initial scraping step (e.g. craigslist.org) to populate the set of URLs or location names we needed for a given source, with this facilitating the creation of all Cronjob manifests programmatically rather than manually ([Scale-Down \(Human Effort\)](#)). Between Helena's parameterization and Kubernetes' easier container deployment controls, we now have a means for quickly deploying scrapers to cover all possible geographic locations and sources of interest from essentially the same scraping framework that we started with back in 2016.

Once our cluster was prototyped, we noticed an unexpected behavior where Zillow containers never reached a completed status, tying up computing resources and preventing proper progression through the full set of Zillow jobs. Thankfully, running the demonstrated program outside of our Kubernetes cluster via the Helena GUI let us look at what was tripping up these scrapers, where it became evident that this issue was driven by ads that loaded slowly or never at all. ([Single-User Mixed Expertise Levels](#)) With this diagnostic information, we solved this issue by modifying the Helena Docker image used for Zillow to incorporate the open-source adblocking extension uBlock Origin ([Integration](#)).

With expanded data collection across locations and sources, we have been able to advance our research about online housing platforms and their role in the housing search process in a number of new ways. First, we used our national data from multiple platforms to assess neighborhood- and market-level variations in the degree to which neighborhoods are represented online [14]. This study shows how more racially and ethnically diverse neighborhoods tend to be underrepresented online on both Craigslist and Apartments.com, though we observe that this lack of coverage is most pronounced for such neighborhoods in already-segregated markets like Chicago rather than more integrated ones like Seattle. We also contributed another policy analysis to HUD's *Cityscape* journal in

which we compare the user interfaces and data of various platforms to investigate the degree to which platforms target different segments of the rental market and facilitate different housing search strategies [8]. The degree to which using a given platform generates different search results and provides qualitatively different information highlights how variations in the online housing search process may lead to distinctly different neighborhood and housing outcomes when households change residences.

Finally, a study we are currently working on investigates the extent to which rental market information is segmented between mainstream information providers (Craigslist, Zillow, Apartments.com) and a platform uniquely focused on low-rent units where housing vouchers can be used (GoSection8). Using American Community Survey data on the distribution of vacant housing units for rent to inform an expected distribution of ads, Figure 1 shows how the neighborhoods underrepresented on Zillow (indicated with hatching) include the historically segregated and structurally disadvantaged neighborhoods of the South and West Sides of Chicago, all while the neighborhoods underrepresented on GoSection8 (indicated in green) are disproportionately in the more affluent and majority non-Latino White areas around the central business district and North Side. Patterns of neighborhood representation for Atlanta, Boston, Seattle and other cities not visualized similarly illustrate how the information about housing that one sees is structured by the platform one visits. While information segmentation has implications for housing search outcomes among the broader public, the findings of this study also have clear policy importance since many public housing authorities use GoSection8 as a source of information about housing opportunities for their households—even if the ads on this platform focus disproportionately on disadvantaged areas where assisted households are already concentrated.

4 DISCUSSION

This case study highlights the promise of deploying novel computing tools within social science collaborations where domain experts may be interested in applying computing methods (e.g., web automation) but lack technical skills to accomplish the particular task by themselves. This computer science–social science collaboration has generated important insights for basic and policy research about housing, while also helping to identify ways to advance the usability and learnability of future automation tools and programming languages. In this collaboration, access to a new human-centered programming tool made it feasible to apply web automation and data scraping within housing research that otherwise depends on secondary data sources for its inputs. We anticipate that this same pattern of joining HCI techniques and social scientists' goals could be repeated across many more fields and subfields within the social sciences, with the same results: advances in HCI, advances in the social science.

Our experience sheds light on important implications for future sociological and policy research where existing data sources suffer from limitations like timeliness or geographic coverage. First, this project shows how adopting new computing tools within social science research does not need to mean becoming a computer scientist first. While the cost-benefit analysis for thoroughly learning

a skill like web automation may be negative when one is only interested in experimenting with it on a small scale, new learnable tools from HCI are giving us additional options. Human-centered automation tools may empower potential users like social scientists to simply try a new idea out before making a decision about whether the return on their investment will be sufficient. Further, starting projects with tools like Helena that facilitate scaling without substantial technical training gives one the means to address broader and more important research questions once an initial idea is validated. Despite the real challenges to incorporating novel computing methods within existing research efforts shown in this study, the collaboration of computer scientists with social scientists affords new directions for computational research that addresses policy and socially-relevant subjects like housing unaffordability and neighborhood inequalities.

For computer scientists, particularly those engaged in human-computer interaction research, this case study highlights ways that computing tools aimed at solving common computing challenges (like web automation) can be designed to enable applications ranging from one-off use to ongoing, relatively large-scale projects with multiple stakeholders. Techniques like programming-by-demonstration can complement users' strengths by taking advantage of users' domain knowledge without demanding extensive computing expertise. More broadly, this case study shows how new human-centered computing tools can be designed around a core prototyping interaction that requires very little onboarding, while still incorporating features that accommodate increasing scale and complexity. Finally, we see the advantages of giving users the flexibility to switch between (i) expert-friendly and program-centric and (ii) beginner-friendly and visual interactions even as they advance in technical knowledge. In our case, we saw this when we needed to return to the GUI to see why Zillow pages never finished loading. Running thousands of scrapers a day from the GUI would be impossible or at least uncomfortable, but returning to the GUI for diagnostics was useful. Allowing users to jump up and down the abstraction stack as they become comfortable with some tasks or as they face new and uncomfortable tasks—this approach lets users achieve scale-up without sacrificing the troubleshooting or debugging capabilities they can get from the beginner-friendly tools.

Overall, this case study contributes an illustrative example of how interdisciplinary collaboration between computer and social scientists is fruitful not only in terms of research outputs but also in societal impact. Research based on data collected with the novel Helena programming tool has informed housing policy at local and national scales, while also contributing to theoretical debates among sociologists about how patterns of residential inequality are reproduced over time. Collaborations like this are even likely to generate entirely new research directions (e.g., online platform segmentation) since research at the intersection of technology with social processes necessarily pushes social scientists to grapple with the role of technology in a world in which we are surrounded by data—and in this case, simply need new tools to collect it.

ACKNOWLEDGMENTS

To Kyle Crowder, Ian Kennedy, Rebecca Walter, Arthur Acolin and the UW graduate students who have helped with this project's development since 2016.

REFERENCES

- [1] Geoff Boeing and Paul Waddell. 2017. New insights into rental housing markets across the United States: Web scraping and analyzing craigslist rental listings. *Journal of Planning Education and Research* 37, 4 (2017), 457–476.
- [2] Sarah Chasins and Rastislav Bodik. 2017. Skip Blocks: Reusing Execution History to Accelerate Web Scripts. *Proc. ACM Program. Lang.* 1, OOPSLA, Article 51 (Oct. 2017), 28 pages. <https://doi.org/10.1145/3133875>
- [3] Sarah Elizabeth Chasins. 2019. *Democratizing Web Automation: Programming for Social Scientists and Other Domain Experts*. Ph.D. Dissertation. UC Berkeley.
- [4] Sarah E. Chasins, Maria Mueller, and Rastislav Bodik. 2018. Rousillon: Scraping Distributed Hierarchical Web Data. In *Proceedings of the 31st Annual ACM Symposium on User Interface Software and Technology* (Berlin, Germany) (UIST '18). Association for Computing Machinery, New York, NY, USA, 963–975. <https://doi.org/10.1145/3242587.3242661>
- [5] Raj Chetty, Nathaniel Hendren, and Lawrence F Katz. 2016. The effects of exposure to better neighborhoods on children: New evidence from the Moving to Opportunity experiment. *American Economic Review* 106, 4 (2016), 855–902.
- [6] Jennie E Christensen. 2020. The Demise of the CFAA in Data Scraping Cases. *Notre Dame JL Ethics & Pub. Pol'y* 34 (2020), 529.
- [7] PostGIS Project Steering Committee. 2021. PostGIS - Spatial and Geographic objects for PostgreSQL. Website.
- [8] Ana Costa, Victoria Sass, Ian Kennedy, Roshni Roy, Rebecca J Walter, Arthur Acolin, Kyle Crowder, Chris Hess, Alex Ramiller, and Sarah Chasins. 2021. Toward a Cross-Platform Framework. *Cityscape* 23, 2 (2021), 327–340.
- [9] Kyle Crowder, Erin Carll, Jerald Herting, and Chris Hess. 2018. Seattle Rental Housing Study.
- [10] Kyle Crowder, Matthew Hall, and Stewart E Tolnay. 2011. Neighborhood immigration and native out-migration. *American sociological review* 76, 1 (2011), 25–47.
- [11] Marcia Fernald et al. 2021. The State of the Nation's Housing. *Joint Center for Housing Studies of Harvard University* (2021).
- [12] Joint Center for Housing Studies of Harvard University. 2021. The Rent Eats First: Rental Housing Unaffordability in the US. (2021).
- [13] John I Gilderbloom, Lin Ye, Matthew J Hanka, and Kareem M Usher. 2009. Intercity rent differentials in the US housing market 2000: Understanding rent variations as a sociological phenomenon. *Journal of urban affairs* 31, 4 (2009), 409–430.
- [14] Chris Hess, Arthur Acolin, Rebecca Walter, Ian Kennedy, Sarah Chasins, and Kyle Crowder. 2021. Searching for housing in the digital age: Neighborhood representation on internet rental housing platforms across space, platform, and metropolitan segregation. *Environment and Planning A: Economy and Space* (2021), 0308518X2111034177.
- [15] Chris Hess, Gregg Colburn, Kyle Crowder, and Ryan Allen. 2020. Racial disparity in exposure to housing cost burden in the United States: 1980–2017. *Housing Studies* (2020), 1–21.
- [16] Christian Hess, Rebecca J Walter, Arthur Acolin, and Sarah Chasins. 2019. Comparing Small Area Fair Market Rents with Other Rental Measures across Diverse Housing Markets. *Cityscape (Washington, DC)* 21, 3 (2019), 159.
- [17] iOpus. 2013. Browser Scripting, Data Extraction and Web Testing by iMacros. <http://www.iopus.com/imacros/>.
- [18] Ian Kennedy, Chris Hess, Amandalynne Paullada, and Sarah Chasins. 2021. Racialized Discourse in Seattle Rental Ad Texts. *Social Forces* 99, 4 (2021), 1432–1456.
- [19] Maria Krysan and Kyle Crowder. 2017. *Cycle of segregation: Social processes and residential stratification*. Russell Sage Foundation.
- [20] Tessa Lau. 2009. Why programming-by-demonstration systems fail: Lessons learned for usable ai. *AI Magazine* 30, 4 (2009), 65–65.
- [21] David Lazer and Jason Radford. 2017. Data ex machina: introduction to big data. *Annual Review of Sociology* 43 (2017), 19–39.
- [22] Vu Le and Sumit Gulwani. 2014. FlashExtract: A Framework for Data Extraction by Examples. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Edinburgh, United Kingdom) (PLDI '14). ACM, New York, NY, USA, 542–553. <https://doi.org/10.1145/2594291.2594333>
- [23] Gilly Leshed, Eben M. Haber, Tara Matthews, and Tessa Lau. 2008. CoScripter: automating & sharing how-to knowledge in the enterprise. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Florence, Italy) (CHI '08). ACM, New York, NY, USA, 1719–1728. <https://doi.org/10.1145/1357054.1357323>
- [24] Henry Lieberman. 2001. *Your wish is my command: Programming by example*. Morgan Kaufmann.
- [25] Henry Lieberman, Fabio Paternò, Markus Klann, and Volker Wulf. 2006. End-user development: An emerging paradigm. In *End user development*. Springer, 1–8.
- [26] Wil Lieberman-Cribbin, Stephanie Tuminello, Raja M Flores, and Emanuela Taioli. 2020. Disparities in COVID-19 testing and positivity in New York City. *American journal of preventive medicine* 59, 3 (2020), 326–332.
- [27] James Lin, Jeffrey Wong, Jeffrey Nichols, Allen Cypher, and Tessa A. Lau. 2009. End-user programming of mashups with Vegemite. In *Proceedings of the 14th international conference on Intelligent user interfaces* (Sanibel Island, Florida, USA) (IUI '09). ACM, New York, NY, USA, 97–106. <https://doi.org/10.1145/1502650.1502667>
- [28] Ann Owens. 2019. Building inequality: Housing segregation and income segregation. *Sociological Science* 6 (2019), 497.
- [29] R Core Team. 2013. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. <http://www.R-project.org/>
- [30] CA: Environmental Systems Research Institute Redlands. 2011. ArcGIS Desktop: Release 10.
- [31] Daniel Schneider and Kristen Harknett. 2019. Consequences of routine work-schedule instability for worker health and well-being. *American Sociological Review* 84, 1 (2019), 82–114.
- [32] Wei Wang, David Rothschild, Sharad Goel, and Andrew Gelman. 2015. Forecasting elections with non-representative polls. *International Journal of Forecasting* 31, 3 (2015), 980–991.